if i want test authentication and authorization broken
what type of input output i want to give all vule from auth both
i want make tool this can test it and make report about bug
all bug from this both tool in auto

If you want to build a **tool that automatically tests for Authentication and Authorization vulnerabilities** and then **generates a report**, you need to design the tool around **inputs, test actions, and outputs (findings)**. I'll break it down clearly so you can implement it.

---

1. Scope of Your Tool

Your tool should detect two categories:

1️⃣ Broken Authentication

Problems in **login, session management, identity verification**

Examples:

- Weak login
- Session hijacking
- Token manipulation
- Password issues

2️⃣ Broken Authorization

Problems in **access control**

Examples:

- IDOR
- Privilege escalation
- Accessing admin endpoints

- Role bypass

These are listed in **OWASP Top 10 vulnerabilities.

---

2. Inputs Your Tool Needs

Your tool must accept these inputs:

Target Information

```
target_url
login_url
api_endpoints
```

Example

```
https://example.com
https://example.com/login
https://example.com/api/*
```

Authentication Data

```
username
password
token
cookies
headers
```

Example

```
username=test
password=test123
Authorization: Bearer TOKEN
```

User Roles (important for auth testing)

```
admin_user
normal_user
guest_user
```

Example

```
admin: admin@test.com
user: user@test.com
guest: none
```

Endpoint List

Your tool must crawl or accept endpoints.

Example

```
/admin
/api/users
/api/user/1
/api/user/2
/dashboard
/settings
```

---

3. Test Payloads Your Tool Should Send

Broken Authentication Tests

1. Credential Stuffing

```
admin:admin
admin:password
test:test
```

---

2. Brute Force

```
password list
common passwords
```

---

3. Session Fixation

Send fixed session cookie:

```
Cookie: session=123456
```

Check if login keeps same session.

---

4. JWT Manipulation

If JWT exists:

Change

```
role:user → role:admin
```

or

```
alg: none
```

---

5. Token Reuse

Use expired token:

```
Authorization: Bearer OLD_TOKEN
```

---

4. Authorization Tests

1. IDOR Test

Change object IDs:

```
/api/user/1
/api/user/2
/api/user/3
```

If user can access other users → vulnerability.

---

2. Privilege Escalation

Normal user accessing admin endpoint:

```
GET /admin
GET /api/admin/users
```

---

3. Method Manipulation

Change HTTP method:

```
GET
POST
PUT
DELETE
```

Example:

```
DELETE /api/user/5
```

---

4. Parameter Tampering

Change role:

```
role=user → role=admin
```

---

5. Tool Workflow

Your tool logic should look like this:

```
1. Crawl target
2. Collect endpoints
3. Detect login
4. Authenticate users
5. Store cookies/tokens
6. Run auth tests
7. Run authorization tests
8. Compare responses
9. Detect anomalies
10. Generate report
```

---

6. Output Your Tool Should Produce

Your tool should generate a **security report**.

Example JSON report:

```
{
 "target":"example.com",
 "vulnerabilities":[
  {
   "type":"IDOR",
   "endpoint":"/api/user/2",
   "severity":"High",
   "description":"User can access other user data"
  },
  {
   "type":"Broken Authentication",
   "endpoint":"/login",
   "severity":"Critical",
   "description":"Weak password accepted"
  }
 ]
}
```

---

7. Report Format (Better)

Include:

```
Vulnerability Name
Severity
```

```
Endpoint
Request
Response
Proof of Concept
Fix Recommendation
```

Example

```
Vulnerability: IDOR
Severity: High

Endpoint:
GET /api/user/2

Issue:
User ID can be modified to access other users.

Recommendation:
Implement proper access control checks.
```

---

8. Technologies You Can Use

To build the tool:

Language

- Python
- Go
- Node.js

Libraries

Python:

```
requests
httpx
jwt
beautifulsoup
aiohttp
```

---

9. Advanced Features

Add:

Automatic Endpoint Discovery

like:

- crawler
- sitemap parser
- JS parser

Differential Response Analysis

Compare:

```
user request response
admin request response
```

Rate limit detection

---

10. Similar Tools (For Inspiration)

Look at:

- Burp Suite
- OWASP ZAP
- Nuclei

---

11. Pro Tip (Important)

For auth testing you must track:

```
cookies
tokens
headers
roles
```

Most vulnerabilities appear when:

```
userA token → access userB data
```

✅ If you want, I can also show:

- **Full architecture of an Automated Auth/Authorization Bug Scanner**
- **Payload list used by professional bug bounty hunters**
- **Python code for building the scanner**
- **200+ auth vulnerability test cases**

Just tell me.

i want all area where i test broken authenticaion and authorization all ares of website
Weak login

Session hijacking

Token manipulation

Password issues
like this
and what i want for test it full how i can
like this give full

To test **Broken Authentication and Broken Authorization across all areas of a website**, you need a **structured checklist of attack surfaces + test methods**. Most security testing standards like **OWASP** and **OWASP Foundation** (OWASP Testing Guide) follow this approach.

Below is a **complete testing map** you can use to design your **automated tool** or manual testing workflow.

---

1. Login System Testing (Authentication Entry Point) 🔐

Areas to Test

- Login page
- API login endpoint
- Mobile login API
- OAuth login
- SSO login

Vulnerabilities

Weak Credentials

```
admin:admin
admin:password
test:test
```

Brute Force

- unlimited login attempts
- no CAPTCHA
- no account lock

User Enumeration

Test responses:

```
Wrong username
Wrong password
```

Different responses = vulnerability.

SQL injection in login

```
' OR 1=1--
admin'--
```

---

2. Password Security Testing 🔑

Areas

- registration
- password reset
- change password
- forgot password

Tests

Weak Password Policy

Try:

```
123456
password
admin123
```

Password Reset Vulnerabilities

Test:

- reset token reuse
- predictable token
- no expiration
- no rate limit

Example:

```
/reset?token=123456
```

Change Password Logic

Check:

```
change password without old password
```

---

3. Session Management Testing 🍪

Session = authentication state.

Areas

```
Cookies
Session ID
Session storage
Headers
```

Vulnerabilities

Session Fixation

1. Set session cookie
2. Login
3. If session remains same → vulnerable

Example cookie:

```
Cookie: sessionid=123456
```

Session Hijacking

Check if:

```
session id predictable
```

Example:

```
sessionid=1001
sessionid=1002
sessionid=1003
```

Session Timeout

Check:

```
session still valid after logout
```

Session in URL

Bad example:

```
example.com/dashboard?sessionid=12345
```

---

4. Token-Based Authentication Testing (JWT / API) 🔑

Modern apps use **JWT tokens**.

Areas

```
Authorization header
API tokens
JWT tokens
OAuth tokens
```

Vulnerabilities

JWT Algorithm None

Change:

```
alg: HS256
```

to

```
alg: none
```

JWT Role Manipulation

Example payload:

```
{
 "user":"test",
 "role":"user"
}
```

Change to:

```
role:admin
```

Expired Token Accepted

Test:

```
exp: 2020
```

Token Leakage

Check if token appears in:

```
URL
logs
localStorage
```

---

5. Multi-Factor Authentication (MFA) Testing 📱

Areas

```
OTP verification
2FA login
email verification
```

Vulnerabilities

OTP brute force

```
000000
111111
123456
```

OTP reuse

Use same OTP again.

Bypass MFA

Skip step:

```
/login → directly /dashboard
```

---

6. Access Control / Authorization Testing 🔐

This is the **largest area**.

Vertical Privilege Escalation

Normal user → admin access

Test:

```
/admin
/admin/users
/admin/settings
```

Horizontal Privilege Escalation

User A → User B data.

Example:

```
/api/user/123
/api/user/124
```

IDOR (Insecure Direct Object Reference)

Test parameters:

```
id
user_id
account_id
order_id
```

Example:

```
/api/orders?user=123
```

Change:

```
user=124
```

---

7. API Authorization Testing ⚙️

APIs often have broken authorization.

Test:

```
GET /api/users
POST /api/users
PUT /api/users/1
DELETE /api/users/1
```

Test Method Manipulation

Example:

```
GET /api/admin/deleteUser
POST /api/admin/deleteUser
```

---

8. File Upload Authorization 📂

Check if normal users can upload restricted files.

Example:

```
upload.php
```

Try:

```
shell.php
admin.php
```

---

9. Direct URL Access Testing 🌐

Try accessing pages directly without login.

Example:

```
/dashboard
/admin
/profile
/settings
```

---

10. Role-Based Access Control (RBAC) 👥

Test roles:

```
guest
user
moderator
```

```
admin
superadmin
```

Test each role against:

```
admin endpoints
user endpoints
payment endpoints
```

---

11. Logout Testing 🚪

Check if:

```
logout actually destroys session
```

Test:

```
logout → back button → dashboard
```

---

12. CORS Authentication Issues 🌍

Misconfigured **CORS** may expose authenticated data.

Check headers:

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

---

13. Cookie Security 🍪

Check cookie flags:

```
Secure
HttpOnly
SameSite
```

Example:

```
Set-Cookie: session=abc123
```

Should include:

```
Secure
HttpOnly
SameSite=Strict
```

---

14. Mobile API Authentication 📱

Mobile apps often expose:

```
api.mobile.example.com
```

Test:

```
token reuse
token leakage
missing auth
```

---

15. Business Logic Authorization 💼

Example:

User should not access:

```
/api/refund
/api/admin/refund
```

---

16. GraphQL Authorization

If GraphQL exists:

```
/graphql
```

Try:

```
query {
 users {
  id
  email
 }
}
```

---

17. Testing Workflow for Your Tool

Your automated tool should do:

```
1. Crawl website
2. Collect endpoints
3. Detect login forms
4. Authenticate users
5. Save tokens/cookies
6. Test session issues
7. Test IDOR
8. Test role escalation
9. Test token manipulation
10. Compare responses
11. Generate report
```

---

18. Inputs for Your Tool

```
Target URL
Login endpoint
Credentials
Role accounts
Token format
API endpoints
```

---

19. Output (Bug Report)

Example:

```
Vulnerability: IDOR
Severity: High

Endpoint:
/api/user/124

Description:
User can access another user's data by changing user ID.

Recommendation:
Implement server-side access control.
```

---

20. If You Want a Professional-Level Tool

I can also show you:

- **150+ Authentication attack test cases**
- **Authorization bypass payload list**
- **How tools like Burp Suite and OWASP ZAP test auth automatically**
- **Architecture for a fully automated Auth vulnerability scanner**

Just tell me and I'll give the **complete blueprint (very powerful)**.

150+ Authentication attack test cases

Below is a **large checklist (150+ test cases)** you can use for **Broken Authentication testing** when building your automated tool. The structure is inspired by testing practices from **OWASP** and the **OWASP Foundation Testing Guide**.

I grouped them so your tool can run **modules**.

---

1. Login Form Security Tests (20 tests) 🔐

1. Login with empty username
2. Login with empty password
3. Login with both fields empty
4. Login with very long username (5000 chars)
5. Login with very long password (5000 chars)
6. Login with SQL injection payload
7. Login with XSS payload
8. Login using unicode characters
9. Login with leading spaces in username
10. Login with trailing spaces in username
11. Login with uppercase vs lowercase username
12. Login using email instead of username
13. Login with invalid email format
14. Login with multiple login attempts quickly
15. Login with different user agents
16. Login from multiple IP addresses simultaneously
17. Login using disabled account
18. Login using deleted account
19. Login using expired account
20. Login with session already authenticated

---

2. Brute Force Protection Tests (15 tests) 🚨

21. Unlimited login attempts
22. Brute force using common passwords
23. Brute force using dictionary attack
24. Login attempts without CAPTCHA
25. CAPTCHA bypass attempt
26. Check account lockout after 5 attempts
27. Check account lockout bypass by IP rotation
28. Check account lockout bypass by username variation
29. Parallel brute force attack
30. Password spraying attack
31. Brute force using API endpoint

32. Brute force using mobile API
33. Check login rate limiting
34. Check if login response time reveals valid username
35. Check brute force detection alerts

---

3. User Enumeration Tests (10 tests) 🔎

36. Different error messages for wrong username
37. Different error messages for wrong password
38. Response time difference
39. Password reset reveals valid email
40. Registration reveals existing email
41. Login response size difference
42. GraphQL user existence check
43. API response reveals username validity
44. Forgot password reveals account existence
45. 2FA page reveals valid users

---

4. Password Policy Tests (15 tests) 🔑

46. Accepts password "123456"
47. Accepts password "password"
48. Accepts password equal to username
49. Accepts short passwords (<6 chars)
50. Accepts password without numbers
51. Accepts password without special characters
52. Accepts reused passwords
53. Allows previously used passwords
54. Allows password identical to email
55. Allows common password dictionary
56. Accepts whitespace passwords
57. Password visible in request logs
58. Password transmitted without encryption
59. Password stored in localStorage

60. Password stored in browser autofill insecurely

---

5. Session Management Tests (20 tests) 🍪

61. Session ID predictable
62. Session ID sequential
63. Session ID exposed in URL
64. Session ID not regenerated after login
65. Session ID reused after logout
66. Session ID accepted after password change
67. Session not invalidated after logout
68. Session active after inactivity
69. Session cookie missing Secure flag
70. Session cookie missing HttpOnly
71. Session cookie missing SameSite
72. Session accessible via JavaScript
73. Session leaked via referrer header
74. Session leaked via logs
75. Multiple sessions allowed simultaneously
76. Session fixation attack
77. Session ID brute force
78. Session reuse across IP addresses
79. Session reuse across browsers
80. Session persists after password reset

---

6. Token Authentication Tests (JWT / API) (20 tests) 🔑

81. JWT algorithm "none" attack
82. JWT signature removed
83. JWT signature changed
84. JWT payload tampering
85. JWT role modification
86. JWT expiration bypass
87. JWT token reuse after logout

88. JWT token reuse after password change
89. JWT token leaked in URL
90. JWT stored in localStorage
91. JWT stored in sessionStorage
92. JWT accessible via JavaScript
93. JWT accepted after expiration
94. JWT missing issuer validation
95. JWT missing audience validation
96. JWT key confusion attack
97. JWT brute force secret key
98. JWT replay attack
99. JWT token accepted across users
100. JWT token accepted from different IP

---

7. Multi-Factor Authentication Tests (15 tests) 📱

101. Login bypass without MFA
102. Access dashboard before MFA verification
103. OTP brute force
104. OTP reuse attack
105. OTP predictable pattern
106. OTP sent via insecure channel
107. OTP unlimited attempts
108. OTP bypass using API call
109. MFA session fixation
110. MFA code leaked in response
111. MFA code accepted multiple times
112. MFA code accepted after expiration
113. Backup codes reusable
114. MFA bypass via mobile API
115. MFA bypass using direct endpoint

---

8. Password Reset Tests (15 tests) 🔄

116. Password reset token predictable
117. Password reset token brute force
118. Password reset token reuse
119. Password reset token not expired
120. Password reset link usable multiple times
121. Password reset without email verification
122. Password reset without OTP verification
123. Password reset using old token
124. Password reset link leakage in referrer
125. Password reset token in URL logs
126. Password reset token not bound to user
127. Password reset token usable across accounts
128. Password reset CSRF attack
129. Password reset bypass using API
130. Password reset token accepted after logout

---

9. Logout Tests (10 tests) 🚪

131. Logout does not invalidate session
132. Back button returns to dashboard after logout
133. Session still active after logout
134. JWT still valid after logout
135. API still accessible after logout
136. Multiple sessions not logged out
137. Logout endpoint missing CSRF protection
138. Logout via GET request allowed
139. Logout response leaks session ID
140. Logout endpoint accessible without authentication

---

10. Authentication Flow Logic Tests (15 tests) 🧠

141. Direct access to dashboard without login
142. Access API endpoints without authentication
143. Access admin panel without login

144. Skip login step using API
145. Skip verification step in auth flow
146. Access authenticated resources via cached page
147. Access authenticated pages via mobile API
148. Authentication bypass via parameter manipulation
149. Authentication bypass via HTTP header manipulation
150. Authentication bypass via cookie tampering
151. Authentication bypass via alternate endpoint
152. Authentication bypass via GraphQL endpoint
153. Authentication bypass via WebSocket
154. Authentication bypass via legacy API
155. Authentication bypass via debug endpoint

Example Output Your Tool Should Generate

```
Vulnerability: JWT Role Manipulation
Severity: Critical

Endpoint:
/api/profile

Proof:
Changed role=user → role=admin in JWT token.

Result:
Admin panel access granted.

Recommendation:
Validate JWT signature and role on server.
```

💡 Tip for building your tool

Your tool should have modules like:

```
auth_login_tests()
auth_session_tests()
auth_token_tests()
```

```
auth_mfa_tests()
auth_reset_tests()
```

✅ If you want, I can also give you:

- **200+ Authorization test cases (even bigger list)**
- **Full architecture for an automated Auth vulnerability scanner**
- **Payload database used by bug bounty hunters**
- **How tools like Burp Suite and OWASP ZAP detect auth bugs automatically**

Just tell me.

200+ Authorization test cases (even bigger list)

Below is a **large checklist (200+ Authorization test cases)** for testing **Broken Authorization / Access Control** across a website or API. These practices follow security guidance from **OWASP** and the **OWASP Foundation Testing Guide**.

You can convert each test case into **automated checks in your security testing tool**.

---

1. Direct Object Reference (IDOR) Tests (40 tests) 🔎

Test by modifying object IDs.

1. Access another user profile by changing `user_id`
2. Access another user order by changing `order_id`
3. Access another user invoice by changing `invoice_id`
4. Access another user's uploaded files
5. Access another user's private messages
6. Access another user's settings page
7. Access another user's payment history
8. Access another user's shopping cart

9. Access another user's wishlist
10. Access another user's notifications
11. Access another user's API key
12. Access another user's support tickets
13. Access another user's documents
14. Access another user's email logs
15. Access another user's activity logs
16. Access another user's profile image
17. Access another user's backup files
18. Access another user's export data
19. Access another user's audit logs
20. Access another user's password reset history
21. Access another user's saved addresses
22. Access another user's transactions
23. Access another user's billing details
24. Access another user's subscription details
25. Access another user's connected apps
26. Access another user's OAuth tokens
27. Access another user's shared files
28. Access another user's internal messages
29. Access another user's draft posts
30. Access another user's unpublished content
31. Access another user's API tokens
32. Access another user's stored credentials
33. Access another user's database export
34. Access another user's system logs
35. Access another user's security settings
36. Access another user's session logs
37. Access another user's admin notes
38. Access another user's customer records
39. Access another user's configuration files
40. Access another user's project data

---

2. Horizontal Privilege Escalation (30 tests) 👥

User A accessing User B data.

41. Modify `user_id` in API request
42. Modify `account_id` parameter
43. Modify `customer_id` parameter
44. Modify `profile_id` parameter
45. Modify `document_id` parameter
46. Change cookie user identifier
47. Change JWT user claim
48. Change session variable user ID
49. Change request body user ID
50. Change hidden form field user ID
51. Change URL path user ID
52. Change GraphQL user ID query
53. Change API request header user ID
54. Change mobile API user ID
55. Change WebSocket request user ID
56. Access another user's private API endpoint
57. Download another user's reports
58. Access another user's uploaded files
59. Access another user's database records
60. Access another user's backup archives
61. Access another user's team resources
62. Access another user's organization data
63. Access another user's shared projects
64. Access another user's private dashboards
65. Access another user's export functionality
66. Access another user's admin logs
67. Access another user's cloud storage
68. Access another user's analytics data
69. Access another user's payment details
70. Access another user's authentication logs

---

3. Vertical Privilege Escalation (Admin Bypass) (30 tests) 🔐

User → Admin access.

71. Access `/admin` page as normal user
72. Access `/admin/users` endpoint
73. Access `/admin/settings` endpoint
74. Access `/admin/deleteUser` API
75. Access `/admin/createUser` API
76. Access `/admin/logs` endpoint
77. Access `/admin/audit` endpoint
78. Access `/admin/export` endpoint
79. Access `/admin/system` endpoint
80. Access `/admin/config` endpoint
81. Access admin GraphQL queries
82. Access admin WebSocket channels
83. Access admin file uploads
84. Access admin backups
85. Access admin database dumps
86. Access admin payment management
87. Access admin billing panel
88. Access admin API tokens
89. Access admin analytics panel
90. Access admin feature flags
91. Access admin debug endpoints
92. Access admin user impersonation
93. Access admin system metrics
94. Access admin log download
95. Access admin message broadcast
96. Access admin server status
97. Access admin service restart endpoint
98. Access admin configuration updates
99. Access admin scheduled tasks
100. Access admin webhook management

4. Role-Based Access Control (RBAC) Tests (30 tests) 🧩

101. User performing moderator actions
102. Guest performing user actions
103. User performing admin actions
104. Moderator performing admin actions
105. Guest accessing private endpoints
106. Guest accessing authenticated APIs
107. User accessing moderator tools
108. User accessing admin dashboard
109. User accessing admin APIs
110. Guest accessing payment endpoints
111. User accessing billing management
112. User accessing system settings
113. User accessing user management
114. User accessing audit logs
115. User accessing admin export tools
116. User modifying system configuration
117. User creating new admin accounts
118. User modifying admin permissions
119. User disabling security features
120. User accessing developer endpoints
121. User accessing internal dashboards
122. User accessing internal APIs
123. User accessing staff-only pages
124. User accessing security configuration
125. User accessing deployment tools
126. User accessing build pipelines
127. User accessing CI/CD endpoints
128. User accessing monitoring tools
129. User accessing alert configuration
130. User accessing feature toggles

---

5. HTTP Method Authorization Tests (20 tests) ⚙️

131. Endpoint protected for GET but not POST
132. Endpoint protected for POST but not PUT
133. Endpoint protected for PUT but not DELETE
134. Endpoint accessible via OPTIONS method
135. Endpoint accessible via PATCH
136. Endpoint accessible via HEAD request
137. Method override header bypass
138. `X-HTTP-Method-Override` bypass
139. Access restricted action via alternate HTTP method
140. Access admin endpoint via POST instead of GET
141. Access admin endpoint via PUT request
142. Access admin endpoint via DELETE request
143. Access admin endpoint via PATCH request
144. Access restricted endpoint via HEAD
145. Access restricted endpoint via OPTIONS
146. Access restricted API via TRACE
147. Access restricted endpoint via CONNECT
148. Access restricted endpoint via WebDAV method
149. Access restricted endpoint via custom method
150. Access restricted endpoint via HTTP/2 request

---

6. Parameter Manipulation Tests (20 tests) 🧪

151. Change `role=user → role=admin`
152. Change `is_admin=false → true`
153. Change `access_level=1 → 10`
154. Change `privilege=user → admin`
155. Change `permission=read → write`
156. Change hidden form role value
157. Change role value in JSON request
158. Change privilege level in cookie
159. Change privilege level in JWT
160. Change permission in API body
161. Change permission in URL parameter

162. Change role in GraphQL mutation
163. Change privilege in WebSocket message
164. Change access level in mobile API
165. Change role in multipart form request
166. Change privilege in file upload request
167. Change role in query string
168. Change role in request headers
169. Change role in XML request
170. Change role in protobuf request

---

7. API Authorization Tests (20 tests) 🔗

171. Access API endpoint without authentication
172. Access API endpoint with invalid token
173. Access API endpoint with expired token
174. Access API endpoint with different user's token
175. Access API endpoint using guest token
176. Access internal API endpoint directly
177. Access admin API endpoint without role check
178. Access API using modified JWT role
179. Access API using stolen token
180. Access API using replay attack
181. Access API via mobile endpoint
182. Access API via GraphQL endpoint
183. Access API via WebSocket endpoint
184. Access API via undocumented endpoint
185. Access API via legacy endpoint
186. Access API via debug endpoint
187. Access API via staging endpoint
188. Access API via internal network endpoint
189. Access API via alternate hostname
190. Access API via CDN endpoint

---

8. Business Logic Authorization Tests (20 tests) 🧠

191. Refund another user's order
192. Cancel another user's subscription
193. Modify another user's order
194. Change another user's password
195. Change another user's email
196. Transfer another user's funds
197. Access another user's private report
198. Modify another user's project
199. Access internal company data
200. Execute admin-only workflow
201. Approve requests without authorization
202. Access restricted workflow steps
203. Trigger privileged background jobs
204. Trigger internal automation scripts
205. Access restricted financial reports

210+ tests total when extended.

Example Vulnerability Output for Your Tool

```
Vulnerability: IDOR
Severity: High

Endpoint:
/api/orders/1024

Test:
Changed order_id from 1023 → 1024

Result:
Order data of another user exposed.

Recommendation:
Implement server-side authorization checks.
```

How Your Automated Tool Should Run These Tests

Workflow:

```
1. Crawl site and collect endpoints
2. Identify authenticated endpoints
3. Capture session/token
4. Modify IDs and roles
5. Send requests automatically
6. Compare responses
7. Detect unauthorized access
8. Generate vulnerability report
```

✅ If you want, I can also give you **very advanced things most bug bounty hunters use**, like:

- **500+ payloads for auth bypass**
- **Complete architecture for an automated Auth/Authorization scanner**
- **AI-powered detection for access control bugs**
- **How tools like Burp Suite and OWASP ZAP detect IDOR automatically**

Just tell me and I'll share the **next level blueprint.** 🚀

Complete architecture for an automated Auth/Authorization scanner

Below is a **complete architecture for building an Automated Authentication & Authorization Vulnerability Scanner**. This design is inspired by how professional tools like **Burp Suite**, **OWASP ZAP**, and **Nuclei** work internally, and follows security guidance from **OWASP**.

I'll explain the **modules, data flow, inputs, engine, and reporting system** so you can implement your tool.

---

1. High-Level Architecture 🧠

Your scanner should have **7 main components**.

```
User Input
   |
   ▼
Target Discovery
   |
   ▼
Authentication Handler
   |
   ▼
Endpoint Collector
   |
   ▼
Authorization Testing Engine
   |
   ▼
Attack Payload Engine
   |
   ▼
Response Analyzer
   |
   ▼
Report Generator
```

---

2. Component 1 — User Input Module 📥

This module receives **all configuration for the scan**.

Inputs Required

```
target_url
login_url
credentials
roles
headers
tokens
```

Example:

```
target: https://example.com
login: https://example.com/login

accounts:
 admin: admin@test.com
 user: user@test.com
```

---

3. Component 2 — Target Discovery Engine 🌐

This module **maps the entire website**.

Methods

1. Web crawler
2. API discovery
3. JavaScript parsing
4. sitemap parsing
5. robots.txt analysis

Collected data:

```
URLs
API endpoints
parameters
HTTP methods
authentication points
```

Example discovered endpoints:

```
/login
/api/users
/api/orders
/admin
/dashboard
```

---

4. Component 3 — Authentication Handler 🔐

This module **handles login and session storage**.

Responsibilities

```
login
store cookies
store tokens
handle sessions
refresh tokens
```

Supported Auth Types

1. Form login
2. API token login
3. JWT authentication
4. OAuth login
5. session cookie authentication

Example stored data:

```
cookie: sessionid=abc123
jwt: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

---

5. Component 4 — Endpoint Collector 📡

This module **organizes endpoints for testing**.

Each endpoint record should contain:

```
endpoint
method
parameters
authentication_required
role_required
```

Example structure:

```
endpoint:
/api/users/{id}

method:
GET
```

```
parameters:
id

auth_required:
true
```

---

6. Component 5 — Attack Payload Engine ⚔️

This module generates **payloads to test vulnerabilities**.

Payload Types

IDOR Payloads

```
id=1
id=2
id=3
```

Role Manipulation

```
role=user → role=admin
is_admin=false → true
```

Token Attacks

```
JWT role change
JWT expiration bypass
JWT algorithm none
```

HTTP Method Manipulation

```
GET
POST
PUT
```

```
DELETE
PATCH
```

Session Attacks

```
session fixation
session reuse
session prediction
```

---

7. Component 6 — Authorization Testing Engine 🔎

This is the **core engine** of your scanner.

It performs tests like:

Horizontal Privilege Escalation

```
User A accessing User B resources
```

Example:

```
/api/user/101
/api/user/102
```

Vertical Privilege Escalation

```
user → admin access
```

Test:

```
/admin
/admin/users
/admin/settings
```

Access Control Bypass

Test:

```
guest access
unauthenticated access
modified roles
```

---

8. Component 7 — Response Analyzer 🧪

This module determines **whether the attack succeeded**.

Detection Techniques

Response Code Analysis

```
200 = allowed
403 = forbidden
401 = unauthorized
```

Response Size Comparison

Example:

```
response userA = 1200 bytes
response userB = 1200 bytes
```

Possible IDOR.

Content Difference

Look for:

```
username
email
account data
```

Behavior Difference

Example:

```
delete request succeeds
```

---

9. Vulnerability Detection Logic 🚨

Example logic for **IDOR detection**.

```
if response_userA == response_userB
and request_userA != request_userB
then vulnerability = IDOR
```

Example for **Admin Bypass**

```
if normal_user_access_admin_endpoint
and response_code == 200
then vulnerability = privilege escalation
```

---

10. Data Storage Layer 🗄️

Store scan data in database.

Example tables:

```
targets
endpoints
requests
responses
vulnerabilities
```

Example vulnerability entry:

```
type: IDOR
endpoint: /api/orders/102
severity: high
```

---

11. Report Generator 📄

Generate reports in formats:

```
JSON
HTML
PDF
Markdown
```

Example report:

```
Vulnerability: Horizontal Privilege Escalation

Endpoint:
/api/user/102

Description:
User A can access User B data.

Severity:
High

Recommendation:
Implement server-side authorization checks.
```

---

12. Scanner Workflow 🔄

Full workflow:

```
1. Receive target
2. Crawl website
3. discover endpoints
4. login using credentials
5. store session
```

```
6. classify endpoints
7. generate attack payloads
8. send test requests
9. analyze responses
10. detect vulnerabilities
11. generate report
```

---

13. Optional Advanced Modules 🚀

Professional scanners add:

1. Role Simulation Engine

```
guest
user
admin
superadmin
```

---

2. AI Response Analysis

Detect subtle differences in responses.

---

3. Rate Limit Bypass Engine

```
IP rotation
header spoofing
proxy pools
```

---

4. Smart IDOR Detection

Find parameters automatically:

```
id
user_id
account_id
order_id
```

14. Recommended Tech Stack ⚙️

Backend

```
Python
Go
Node.js
```

Python libraries:

```
requests
httpx
aiohttp
jwt
beautifulsoup
```

Database

```
PostgreSQL
MongoDB
SQLite
```

Queue System

```
Redis
RabbitMQ
```

---

15. Example Tool Architecture Diagram

```
+----------------+
|  User Input    |
+--------+-------+
         |
         ▼
+----------------+
```

```
| Target Crawler |
+--------+-------+
         |
         ▼
+--------------------+
| Endpoint Collector |
+--------+-----------+
         |
         ▼
+--------------------+
| Authentication     |
| Handler            |
+--------+-----------+
         |
         ▼
+--------------------+
| Attack Engine      |
+--------+-----------+
         |
         ▼
+--------------------+
| Response Analyzer  |
+--------+-----------+
         |
         ▼
+--------------------+
| Report Generator   |
+--------------------+
```

✅ If you want, I can also show you **very powerful next-level things** used by elite security researchers:

- **500+ payloads for auth bypass attacks**
- **How to detect IDOR automatically (advanced logic)**
- **How to scan APIs, GraphQL, and mobile apps**
- **Full Python code to build this scanner**

Just tell me and I'll give you the **complete advanced blueprint.** 🚀

my this tool is full auto matick detect all this thing i want make like this full auto

If you want to build a **fully automatic tool that detects Authentication & Authorization vulnerabilities**, you need to design it so it **discovers everything itself** (login, endpoints, roles, tokens, parameters) and then **tests them automatically**.

Professional scanners like **Burp Suite**, **OWASP ZAP**, and **Nuclei** work with this type of pipeline, based on practices from **OWASP**.

Below is a **practical blueprint for a FULLY AUTOMATED Auth/AuthZ scanner**.

---

1. Fully Automated Scanner Workflow ⚙️

Your tool should run like this automatically:

```
Target URL
   │
   ▼
Website Crawler
   │
   ▼
Endpoint Discovery
   │
   ▼
Authentication Detection
   │
   ▼
Session / Token Extraction
   │
   ▼
Role Discovery
   │
   ▼
Authorization Attack Engine
   │
   ▼
Response Analyzer
   │
```

```
   ▼
Vulnerability Detection
   |
   ▼
Report Generator
```

User only provides:

```
target_url
(optional credentials)
```

Everything else is automatic.

---

2. Automatic Website Crawler 🌐

Your tool must first **map the entire site**.

It should collect:

```
URLs
API endpoints
Forms
Parameters
JavaScript endpoints
GraphQL endpoints
```

Sources:

1. HTML links
2. forms
3. JavaScript files
4. API responses
5. sitemap.xml
6. robots.txt

Example discovered endpoints:

```
/login
/api/users
/api/orders
```

```
/profile
/admin
```

---

3. Automatic Authentication Detection 🔐

Your tool should automatically detect **login mechanisms**.

Look for:

Login Forms

Fields like:

```
username
email
password
login
```

API Login

Look for:

```
POST /api/login
POST /auth/login
POST /api/token
```

Token Authentication

Detect headers like:

```
Authorization: Bearer TOKEN
```

JWT Detection

Check if token format is:

```
xxxxx.yyyyy.zzzzz
```

---

4. Automatic Session Extraction 🍪

After login detection your tool should capture:

```
cookies
session id
JWT tokens
API tokens
headers
```

Example captured session:

```
Cookie: sessionid=abc123
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

---

5. Automatic Role Discovery 👥

Your scanner should try to **identify roles automatically**.

Techniques:

Check JWT payload

Example:

```
{
 "user":"john",
 "role":"user"
}
```

Possible roles:

```
user
admin
moderator
guest
```

Test admin endpoints

Try accessing:

```
/admin
/admin/users
/admin/settings
```

Look for role parameters

Example:

```
role=user
is_admin=false
permission=read
```

---

6. Automatic Parameter Discovery 🔎

Your scanner must identify **sensitive parameters**.

Examples:

```
id
user_id
account_id
order_id
invoice_id
```

Sources:

```
URL
JSON body
form inputs
GraphQL queries
```

Example:

```
/api/user?id=123
```

---

7. Authorization Attack Engine ⚔️

This module performs attacks automatically.

---

1. IDOR Testing

Modify IDs automatically:

```
/api/user/101
/api/user/102
/api/user/103
```

Compare responses.

---

2. Horizontal Privilege Escalation

Use same session but change user ID.

Example:

```
/api/orders?user_id=200
```

---

3. Vertical Privilege Escalation

Normal user trying admin endpoints:

```
/admin
/admin/settings
/admin/users
```

---

4. Role Manipulation

Modify parameters:

```
role=user → admin
is_admin=false → true
```

---

5. Token Manipulation

Modify JWT payload:

```
role:user → role:admin
```

---

6. HTTP Method Manipulation

Try:

```
GET
POST
PUT
DELETE
PATCH
```

Example:

```
DELETE /api/user/101
```

---

8. Smart Response Analyzer 🧠

Your tool must detect vulnerabilities automatically.

Detection techniques:

Response Code Analysis

```
401 unauthorized
403 forbidden
```

```
200 success
```

If:

```
normal user → admin endpoint → 200
```

Then vulnerability.

Response Length Comparison

Example:

```
response1 = 1200 bytes
response2 = 1200 bytes
```

Possible IDOR.

Data Pattern Detection

Look for:

```
email
username
account data
orders
```

---

9. Automatic Vulnerability Detection 🚨

Example detection logic.

IDOR Logic

```
if parameter_changed
and response_data_different_user
then IDOR vulnerability
```

Admin Bypass Logic

```
if normal_user_access_admin_endpoint
and response_code == 200
then privilege escalation
```

Token Manipulation Logic

```
if modified_token_grants_admin_access
then broken authentication
```

---

10. Automated Report Generation 📄

Your tool should output a report automatically.

Example:

```
Target: example.com

Vulnerability: IDOR
Severity: High

Endpoint:
/api/orders/1024

Proof:
Changed order ID from 1023 to 1024.

Result:
Another user's order exposed.
```

---

11. Performance Optimization 🚀

Because scanning thousands of endpoints is heavy.

Use:

```
async requests
multi-threading
queue workers
```

Example architecture:

```
crawler workers
attack workers
analysis workers
```

---

12. Recommended Tech Stack ⚙️

Best stack for this tool.

Language

```
Python
```

Libraries:

```
httpx
aiohttp
requests
jwt
beautifulsoup
```

Storage

```
SQLite
PostgreSQL
```

Queue

```
Redis
Celery
```

13. Advanced Features (Next Level) 🧠

If you want your tool to be **very powerful**:

AI Endpoint Classification

Detect sensitive endpoints automatically.

Smart IDOR detection

Detect numeric patterns automatically.

Example:

```
user_id
order_id
transaction_id
```

Automatic API discovery

Parse:

```
JavaScript
GraphQL
Swagger
OpenAPI
```

---

14. Final Real Architecture

```
+----------------+
|   Target URL   |
+--------+-------+
         |
         ▼
+----------------+
|   Web Crawler  |
+--------+-------+
```

```
                  |
                  ▼
        +----------------+
        | Endpoint Finder|
        +--------+-------+
                  |
                  ▼
        +----------------+
        | Auth Detector  |
        +--------+-------+
                  |
                  ▼
        +----------------+
        | Session Engine |
        +--------+-------+
                  |
                  ▼
        +----------------+
        | Attack Engine  |
        +--------+-------+
                  |
                  ▼
        +----------------+
        | Response AI    |
        +--------+-------+
                  |
                  ▼
        +----------------+
        | Report Engine  |
        +----------------+
```

✅ If you want, I can also show you **extremely advanced things needed to make this tool really powerful**, like:

- **How to automatically detect IDOR without knowing parameters**
- **How elite researchers find auth bugs in minutes**
- **500+ auth bypass payloads**
- **Full Python starter code for this scanner**

Just tell me and I'll show you the **real pro-level design.** 🚀

How elite researchers find auth bugs in minutes

Elite security researchers (top bug bounty hunters and pentesters) can find **Authentication & Authorization bugs in minutes** because they use **fast methodology + automation + pattern recognition**. Many follow workflows inspired by guidance from **OWASP** and tools like **Burp Suite**, **OWASP ZAP**, and **Nuclei**.

Below is the **real workflow used by elite researchers**. 🚀

---

1. First Step: Map the Entire Application Quickly 🗺️

Experts **never start attacking immediately**.
First they discover **all endpoints and parameters**.

They collect:

```
URLs
API endpoints
parameters
roles
tokens
```

Tools / methods

- crawler
- proxy capture
- JS file parsing
- API documentation

Example endpoints discovered:

```
/login
/api/users
/api/orders
/admin
/api/admin/users
```

The faster you map endpoints, the faster you find auth bugs.

---

2. Identify Authentication Mechanism 🔐

Researchers immediately identify **how authentication works**.

Look for:

```
session cookies
JWT tokens
API tokens
OAuth
```

Example request:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

Then decode JWT to see:

```
user
role
permissions
```

If role is inside token → **very interesting target**.

---

3. Look for Authorization Parameters 🔎

Experts immediately search for **sensitive parameters**.

Common ones:

```
id
user_id
account_id
order_id
invoice_id
```

Example:

```
GET /api/orders?id=1201
```

First thing they try:

```
id=1202
```

This alone finds **many IDOR bugs**.

---

4. Check Role-Based Endpoints 👑

Elite researchers immediately try accessing **admin endpoints**.

Example:

```
/admin
/admin/users
/admin/settings
```

Using a normal user session.

If response:

```
200 OK
```

→ **critical privilege escalation**.

---

5. Parameter Tampering 🧪

They modify parameters quickly.

Example request:

```
{
 "role":"user"
}
```

Change to:

```
{
 "role":"admin"
}
```

Also try:

```
is_admin=true
access_level=10
permission=all
```

Sometimes developers trust these values.

---

6. HTTP Method Manipulation ⚙️

Many APIs protect only one method.

Example:

```
GET /api/admin/deleteUser
```

Researcher tries:

```
POST /api/admin/deleteUser
DELETE /api/admin/deleteUser
```

Sometimes this **bypasses authorization**.

---

7. Horizontal Privilege Escalation 👥

Classic bug.

User A accessing User B data.

Example:

```
GET /api/user/123
```

Change:

```
/api/user/124
```

If it returns another user's data → **IDOR vulnerability**.

---

8. Test APIs Directly ⚡

Researchers don't rely on UI.

They go directly to **API endpoints**.

Example:

```
/api/users
/api/orders
/api/payments
```

UI might block something but API may allow it.

---

9. Test JWT Manipulation 🔑

If JWT exists, researchers immediately test:

Change role

```
"user":"john",
"role":"admin"
```

Expiration bypass

```
exp: 9999999999
```

Algorithm none

Change:

```
alg: HS256
```

to

```
alg: none
```

---

10. Look for Business Logic Bugs 🧠

These are **very valuable vulnerabilities**.

Example:

```
/api/refund
/api/cancelOrder
```

Try:

```
refund another user's order
cancel another user's subscription
```

These are often missed by automated scanners.

---

11. Compare Responses Quickly 📊

Experts compare responses like this:

```
User A request
User B request
```

If responses look similar → possible vulnerability.

Example:

```
response size = 1250 bytes
response size = 1250 bytes
```

Then inspect deeper.

12. Automation + Manual Hybrid 🤖

Elite researchers combine:

Automation

Scripts scanning endpoints.

Manual logic

Human thinking for business logic bugs.

---

13. Use Fast Payload Lists ⚔️

Experts have payload lists ready.

Example role payloads:

```
admin
administrator
superadmin
root
staff
```

Example boolean payloads:

```
true
1
yes
on
```

---

14. Focus on High-Risk Areas 🎯

Elite researchers don't test everything randomly.

They prioritize:

Most vulnerable areas

```
admin panels
billing
payments
account settings
file downloads
API endpoints
```

These areas frequently contain auth bugs.

---

15. Pattern Recognition 🧠

Experienced researchers recognize patterns quickly.

Example patterns:

```
/api/user/{id}
/api/order/{id}
/api/account/{id}
```

Immediately test **IDOR**.

---

16. Test Mobile APIs 📱

Mobile APIs often have weak auth.

Example:

```
api.mobile.example.com
```

Researchers inspect mobile apps and extract hidden endpoints.

---

17. Test GraphQL APIs

GraphQL often leaks data.

Example query:

```
query {
 users {
  id
  email
 }
}
```

Sometimes authorization is missing.

---

18. Build Custom Scripts ⚡

Elite researchers often create **small automation scripts**.

Example script tasks:

```
change user IDs
send thousands of requests
compare responses
```

This finds IDOR vulnerabilities fast.

---

19. Use Proxy Tools Properly

The most common tool is:

- intercept requests
- modify parameters
- replay requests

Professional tools used:

- Burp Suite
- OWASP ZAP

---

20. Why Experts Find Bugs in Minutes

Because they:

```
know common patterns
know where to look
automate repetitive tests
focus on high-risk endpoints
```

Beginners search randomly.

Experts search **strategically**.

✅ If you're building your **automated Auth/AuthZ scanner**, the next powerful thing to learn is:

- **How to automatically detect IDOR vulnerabilities (very advanced)**
- **500+ payloads used by top bug bounty hunters**
- **How to build a smart response comparison engine**

If you want, I can show you **the exact algorithm used to detect IDOR automatically**, which is the **hardest part of building your tool**. 🚀